

This homework has 9 questions, for a total of 100 points and 3 extra-credit points.

Unless otherwise stated, each question requires *clear*, *logically correct*, and *sufficient* justification to convince the reader.

We strongly encourage you to typeset your solutions in L^AT_EX.

If you collaborated with someone, you must state their name(s). You must *write your own solution* for all problems and *may not use any other student's write-up or any AI tools*.

0. **Before you start; before you submit.**

If applicable, state the name(s) and username(s) of your collaborator(s).

Solution: No collaborators

- (10 pts) 1. **Self assessment.** Carefully read and understand the posted solutions to the previous homework. Identify one part for which your own solution has the most room for improvement (e.g., has unsound reasoning, doesn't show what was required, could be significantly clearer or better organized, etc.). Copy or screenshot this solution, then in a few sentences, explain what was deficient and how it could be fixed.

(Alternatively, if you think one of your solutions is significantly *better* than the posted one, copy it here and explain why you think it is better.)

If you didn't do last week's homework, choose a problem from it that looks challenging to you, and in a few sentences, explain the key ideas behind its solution in your own words.

Solution:

For Question 3, my correctness argument could be improved because I did not fully justify that every recursive subarray contains a pit. I only considered the boundary near the midpoint and did not account for the other boundary of the recursive interval. The posted solution fixes this by explicitly maintaining the preconditions $M[start - 1] \geq M[start]$ and $M[finish] \leq M[finish + 1]$. These conditions are preserved in every recursive call, making the correctness proof complete. In the future, I should state and maintain such invariants explicitly in my correctness arguments.

2. **Counting strings!** Let $C(n)$ denote the number of binary strings of length n that have no consecutive occurrences of 1. For example, $C(3) = 5$; we see this by listing all binary strings of length 3 and checking that only the strings 000, 001, 010, 100, and 101 have no consecutive 1s.

- (2 pts) (a) Compute $C(5)$.

Solution: There are 13 valid strings of length 5:

00000 00001 00010 00100 00101
01000 01001 01010 10000 10001
10010 10100 10101

None of them contains 11, and any other length-five binary string does contain consecutive 1s. Therefore

$$C(5) = 13.$$

- (4 pts) (b) Derive, with justification, a recurrence relation for $C(n)$. Remember to include appropriate base case(s).

Hint: Consider two cases depending on whether the first character is a 0 or a 1.

Solution: Look at the first bit of a valid length- n string.

If it is 0, removing that 0 leaves any valid string of length $n - 1$. There are $C(n - 1)$ choices. Conversely, putting a 0 in front of any valid length- $n - 1$ string still gives a valid string, so this case really has exactly $C(n - 1)$ strings.

If the first bit is 1, the next bit has to be 0. So the string begins with 10. After removing these two bits, what remains can be any valid string of length $n - 2$, which gives $C(n - 2)$ choices. Again, prefixing 10 to a valid shorter string never creates a pair of consecutive 1s.

The two cases do not overlap, since a string cannot start with both 0 and 1, and they cover every valid string. Hence, for $n \geq 2$,

$$C(n) = C(n - 1) + C(n - 2).$$

The starting values are

$$C(0) = 1 \quad \text{and} \quad C(1) = 2.$$

The first value counts the empty string. For length 1, both 0 and 1 are allowed.

- (3 pts) (c) Translate your recurrence into pseudocode that is *top-down recursive, without memoization*. You may use [Algorithm 26](#) in the course notes as an example. (Note that this algorithm runs in time exponential in the input value n .)

Finally, show a *lower bound* on the runtime of the algorithm, which establishes that the runtime is exponential in n .

(Note that since the input is n , the input *size* is $\Theta(\log n)$, so this exponential-in- n algorithm runs in *double exponential* time with respect to the input size, i.e. $a^{b^{\log n}}$ for some constants a and b .)

Solution: A direct top-down implementation is:

CountStrings(n):

 if $n = 0$:

 return 1

 if $n = 1$:

 return 2

 return CountStrings($n - 1$) + CountStrings($n - 2$)

Let $R(n)$ be the number of calls made by **CountStrings(n)**. For $n \geq 2$, the function makes one recursive call with input $n - 1$, one with input $n - 2$, and also spends one call on the current input. That is why

$$R(n) = R(n - 1) + R(n - 2) + 1 \geq R(n - 1) + R(n - 2).$$

The number of calls is nondecreasing with the input: the recursive call tree for a larger argument contains the same kind of work as the tree for a smaller argument. Thus

$R(n-1) \geq R(n-2)$, and

$$R(n) \geq 2R(n-2).$$

Applying this inequality repeatedly gives

$$R(n) \geq 2R(n-2) \geq 2^2 R(n-4) \geq \dots \geq 2^{\lfloor n/2 \rfloor} R(n-2\lfloor n/2 \rfloor).$$

The last factor is $R(0)$ or $R(1)$, which is 1. Therefore

$$R(n) \geq 2^{\lfloor n/2 \rfloor} = \Omega((\sqrt{2})^n).$$

Every call takes at least constant time, so the running time is exponential in n . If the binary input has $b = \Theta(\log n)$ bits, this lower bound is $2^{2^{\Theta(b)}}$, which is double exponential in the input size.

- (3 pts) (d) Translate your recurrence into dynamic-programming pseudocode. ([Algorithm 28](#) in the course notes is a good model.)

In this and subsequent DP questions, credit for pseudocode will be determined based only on how “faithful” (i.e., properly derived) it is to your recurrence—regardless of whether the recurrence itself is correct. (So, pseudocode that isn’t faithful to your recurrence will not get full credit—but pseudocode that properly matches an incorrect recurrence *will* get full credit.)

Additionally, for all DP questions, your pseudocode should be written in the “bottom-up” form, rather than top-down memoization.

Analyze the running time of your code, as a function of n . Note that unlike most problems, you are *not* being asked to write the running time in terms of the input size (n is the input value, not the input size.) You may use the fact (without proof) that $C(n)$ has $\Theta(n)$ bits, so adding together two entries of the dynamic programming table takes $O(n)$ time.

Solution: The bottom-up version is:

```
CountStringsDP(n):
    if n = 0:
        return 1
    C[0] = 1
    C[1] = 2
    for j = 2 to n:
        C[j] = C[j - 1] + C[j - 2]
    return C[n]
```

The loop computes each value only once, and the values needed on the right side have already been computed. The entry $C[j]$ has $\Theta(j)$ bits, so adding $C[j-1]$ and $C[j-2]$ takes $O(j)$ time. Hence

$$\sum_{j=2}^n O(j) = O\left(\sum_{j=2}^n j\right) = O\left(\frac{n(n+1)}{2} - 1\right) = O(n^2).$$

The table contains $n+1$ entries, and storing all of their bits uses $O(n^2)$ space.

3. Seems Familiar?

In this course, pattern matching is a crucial skill. Many problems you will see on homeworks and exams are, at their cores, similar to those you have previously solved in lecture or discussion. Noticing these similarities allows you to use familiar strategies and techniques to solve new problems effectively.

In this problem, you are given 3 dynamic programming problems below. For each of the following parts, you are given another problem that is the same as one of aforementioned problems and can be solved with same recurrence.

For each part, state and briefly justify which problem it is a reformulation of. Explain which aspects of the two problems correspond to each other.

You do not need to provide the recurrence (though you may want to in order to justify your solution).

- **Knapsack Variant:** Given n items where each item i has an associated value v_i and weight w_i , and no two weights are the same, find a subset of items with maximum possible value, whose total weight is at most C .
- **Coin Change:** Given a list of coins $c = [c_1, \dots, c_k]$, where c_i is the value of coin i , and a target amount n , find the number of different collections of coins that have a total value of n .
- **Stair Climbing:** Given a staircase with n stairs, determine the number of distinct ways a person starting at the bottom can reach the top if each time they take a step they can advance either 1 or 2 stairs.

(4 pts)

- (a) **Pipe Cutting:** Given a pipe of length L , and a list of pipe lengths with associated prices, find the maximum revenue obtainable by cutting the pipe into pieces, and selling the pieces (discarding any leftover pipe), under the constraint that you may only sell each length of pipe a maximum of one time.

Solution: This is a reformulation of the *Knapsack Variant*. Treat each available pipe length as one item: its length is the item's weight, its associated selling price is the item's value, and the original pipe length L is the knapsack capacity. Choosing a set of sellable pipe lengths whose total length is at most L is exactly choosing a subset of items of total weight at most the capacity. The restriction that each length may be sold at most once is the same as the 0–1 knapsack restriction. Discarding leftover pipe corresponds to not needing to use all capacity.

If a pipe length ℓ_r has price q_r , then the knapsack state consisting of the first r lengths and remaining capacity x corresponds exactly to considering the first r sellable piece types while at most x units of the original pipe may still be used. Taking an item corresponds to cutting and selling one piece of that length; skipping it corresponds to never selling that length. This also explains why the no-repeated-length restriction is essential to the match.

(4 pts)

- (b) **Domino Tiling:** Given a $2 \times n$ board, determine the number of ways to completely tile the board using 2×1 dominoes, which can be placed either vertically or horizontally. Every square of the board must be covered by precisely one domino.

Solution: Look at the leftmost two columns that have not yet been tiled. If we place a vertical domino there, the remaining board is a $2 \times (n - 1)$ board. This is the same as taking one stair.

The other possibility starts with a horizontal domino. That domino covers the top square of the first column and the top square of the second column. The bottom square of the first column is still uncovered, so it must be covered by a second horizontal domino. Thus horizontal dominoes always occur as a pair, and the pair leaves a $2 \times (n - 2)$ board. This is the same as taking two stairs.

Therefore the two problems have the same choices at every stage and hence the same recurrence and base cases.

4. Minimum Subarray Product

Given an array $A[1, \dots, n]$ of $n \geq 1$ positive real numbers, we are interested in finding the smallest product of any nonempty contiguous subarray of A , i.e.,

$$\min\{A[i]A[i+1] \cdots A[j] : 1 \leq i \leq j \leq n\}.$$

- (8 pts) (a) In this problem you may assume for simplicity that multiplying a pair of real numbers can be done in $O(1)$ time. Give a recurrence relation, including base case(s), that would yield an $O(n)$ -time dynamic-programming algorithm for the smallest product problem. Briefly justify why your recurrence relation is correct. You will analyze the running time of the resulting algorithm in part (b).

Hint: The longest increasing subsequence recurrence might give you some inspiration (but this recurrence should be simpler than that one).

Solution: For $1 \leq i \leq n$, let $E(i)$ be the smallest product of a nonempty contiguous subarray that ends exactly at position i . For $i = 1$, the only nonempty subarray ending at position 1 is the one-element subarray $[1, 1]$, so

$$E(1) = A[1],$$

and, for $i \geq 2$,

$$E(i) = \min\{A[i], E(i-1)A[i]\}.$$

The desired answer is $\min_{1 \leq i \leq n} E(i)$.

Here is why the recurrence works: take any nonempty subarray ending at i . It is either the one-element subarray $[i, i]$, whose product is $A[i]$, or it has length at least two. In the latter case, removing its final element leaves a nonempty subarray ending at $i - 1$, and its product is that earlier product times $A[i]$. Since every $A[i]$ is positive, multiplication by $A[i]$ preserves order, so the smallest product in the latter case is exactly $E(i - 1)A[i]$. The alternatives are exhaustive, proving the recurrence. Every nonempty contiguous subarray has a unique ending position, so minimizing $E(i)$ over all i gives the global minimum.

After computing $E(i)$, it is important that the state is restricted to subarrays ending at i , rather than arbitrary subarrays among the first i positions. This restriction makes

the transition local: appending $A[i]$ to the best subarray ending at $i - 1$ is the only way to obtain a longer subarray ending at i . Positivity is used precisely here. If negative values were allowed, multiplying by $A[i]$ could reverse inequalities and one would need to track additional information.

- (6 pts) (b) Give pseudocode implementing an $O(n)$ -time dynamic programming solution to the smallest product problem, and justify the running time. Recall that for simplicity, you may assume that multiplying a pair of real numbers can be done in $O(1)$ time. For all DP problems, remember to refer to the “DP Recipe” from lecture and discussion.

Solution: The recurrence can be implemented while storing only the previous table entry:

```
MinimumSubarrayProduct(A[1..n]):
    bestEndingHere = A[1]
    answer = A[1]
    for i = 2 to n:
        bestEndingHere = min(A[i], bestEndingHere * A[i])
        answer = min(answer, bestEndingHere)
    return answer
```

At iteration i , **bestEndingHere** equals $E(i)$, and **answer** equals $\min_{1 \leq j \leq i} E(j)$. Each iteration uses a constant number of real multiplications, comparisons, and assignments, each of which takes $O(1)$ time. Therefore the running time is $O(n)$ and the extra space is $O(1)$.

Before the loop, the two variables equal $E(1)$ and the best product among subarrays ending in the first position. Assuming the invariant before iteration i , the first assignment applies the recurrence and makes **bestEndingHere** equal to $E(i)$; the second assignment compares this new candidate with every previous ending position. Hence the invariant holds after the iteration, and at $i = n$ the returned value is the required global minimum.

5. Sugar Rush!

You and a friend have won first prize at a Halloween costume party (due to your brilliant Alan Turing and Alonzo Church costumes): a bucket of several pieces of candy, each having some amount of sugar. You and your friend plan to celebrate your win by staying up late and eating all of the candy, but first you need to divide it between yourselves. The problem is that if you and your friend consume different amounts of sugar, the one who eats less sugar will crash first, leaving the other person to stay up alone. You need a way to divide the candy between the two of you so that you both eat the same amount of sugar, and crash at the same time.

In this problem you will give a dynamic-programming algorithm that, given an array $S = [s_1, s_2, \dots, s_n]$ where the i th piece of candy has a non-negative integer s_i grams of sugar, either divides the pieces into two ‘piles’ that have equal amounts of sugar, or (correctly) says that no such piles exist. (Each piece must be in exactly one of the piles.)

- (10 pts) (a) For a given array S , define $P(c, i)$ to be **true** if there exists a pile drawn from the first i pieces of candy having a total of c grams of sugar, and **false** otherwise. Derive, with

justification, a recurrence relation for $P(c, i)$. Remember to include appropriate base case(s).

When writing recurrences with Boolean values, you may use operators such as $\wedge$ (and), $\vee$ (or), and $\neg$ (not).

Solution: Let $T = \sum_{r=1}^n s_r$. For $0 \leq c \leq T$, define $P(c, i)$ as in the problem. The base cases are

$$P(0, 0) = \text{true}, \quad P(c, 0) = \text{false} \quad \text{for } 1 \leq c \leq T,$$

and, equivalently, $P(c, i) = \text{false}$ whenever $c < 0$. The first base value is true because the empty set has total 0. With no pieces available, every positive total is impossible, which explains the false values in the second base case. For $i \geq 1$,

$$P(c, i) = P(c, i - 1) \vee (c \geq s_i \wedge P(c - s_i, i - 1)).$$

To see why this works, consider a subset of the first i pieces with total sugar c . Either it does not contain piece i , which is possible exactly when $P(c, i - 1)$ is true, or it does contain piece i . In the latter case, removing piece i leaves a subset of the first $i - 1$ pieces totaling $c - s_i$, which is possible exactly when $c \geq s_i$ and $P(c - s_i, i - 1)$ is true. Those are the only two cases, and they do not overlap, so the recurrence follows.

The second coordinate i means that each of the first i candy pieces has been given a yes/no decision: either it is in the candidate pile or it is not. Thus every subset of the first i pieces appears in exactly one of the two recurrence branches. Because the array entries are Boolean, the recurrence asks only whether at least one such subset exists; it does not need to count how many do.

- (3 pts) (b) Briefly state, with justification, the values of c and i to plug into $P(c, i)$, to determine if it is possible to split the candy into two piles having equal amounts of sugar.

Solution: A split into two equal-sugar piles is possible exactly when T is even and

$$P(T/2, n) = \text{true}.$$

Indeed, a subset of all n pieces with total $T/2$ can be used as one pile; its complement has total $T - T/2 = T/2$ and is the other pile. Conversely, any equal split has one pile totaling $T/2$, so it makes this table entry true. If T is odd, no two integer pile sums can both equal $T/2$.

If two piles have equal integer sugar totals, their common total must be half of T , which first forces T to be even. Moreover, one of those piles is literally a subset of the first n pieces of total $T/2$, so $P(T/2, n)$ must be true. This proves the reverse implication in addition to the construction already given.

- (5 pts) (c) Give pseudocode for a dynamic-programming algorithm that determines whether the candy can be divided into two piles that have the same amount of sugar. Analyze the running time.

Note that this is not an efficient algorithm (in fact, there is no known efficient algorithm for

this problem!). However, if each s_i is upper bounded by a constant, then your algorithm should be efficient.

Solution: The following bottom-up algorithm is a direct implementation of the recurrence. It only needs columns 0 through $T/2$, because larger subset sums are irrelevant to deciding whether an equal split exists.

EqualSugarSplit($S[1..n]$):

```

    T = sum of all entries of S
    if T is odd:
        return false
    H = T / 2

    P[0, 0] = true
    for c = 1 to H:
        P[c, 0] = false

    for i = 1 to n:
        P[0, i] = true
        for c = 1 to H:
            P[c, i] = P[c, i - 1]
            if c >= S[i]:
                P[c, i] = P[c, i] OR P[c - S[i], i - 1]
    return P[H, n]
```

There are $(n+1)(H+1)$ table entries and each is filled in $O(1)$ time. Thus the running time is $O(nH) = O(nT)$, and the full table uses $O(nT)$ space. If every s_i is bounded by a constant, then $T = O(n)$, so the running time becomes $O(n^2)$.

Every entry in column i references only column $i - 1$, never another entry in column i . Therefore the inner loop may process capacities in either increasing or decreasing order when the full two-dimensional table is used. The assignment $P[0, i] = \text{true}$ represents the empty subset, which has sugar total zero regardless of how many pieces are available. All other entries in column zero are false because no positive amount can be formed with no pieces.

- (4 pts) (d) Not eating the candy is not an option! Given the completed DP table from the previous part, describe in clear English how you would find the *smallest possible difference* in sugar between two piles.

Solution: Let $T = \sum_{r=1}^n s_r$. Inspect the final row $P(c, n)$ for $c = \lfloor T/2 \rfloor, \lfloor T/2 \rfloor - 1, \dots, 0$, and let c^* be the first value for which $P(c^*, n)$ is true. One pile can have c^* grams and the other pile, its complement, has $T - c^*$ grams. Their difference is

$$(T - c^*) - c^* = T - 2c^*.$$

Every two-pile division corresponds to some true entry $P(c, n)$; replacing $c > T/2$ by its complement gives the same division with a sum at most $T/2$. Among values at most $T/2$, $T - 2c$ is minimized by the largest feasible c . Therefore this procedure finds the smallest possible difference.

A true entry $P(c, n)$ gives a subset of sugar c and therefore gives the complementary pile automatically; no additional search over partitions is necessary. Scanning downward starts at the point closest to half the total. Once the first true entry is found, every later candidate has a smaller c and hence a weakly larger imbalance, so it cannot improve the answer.

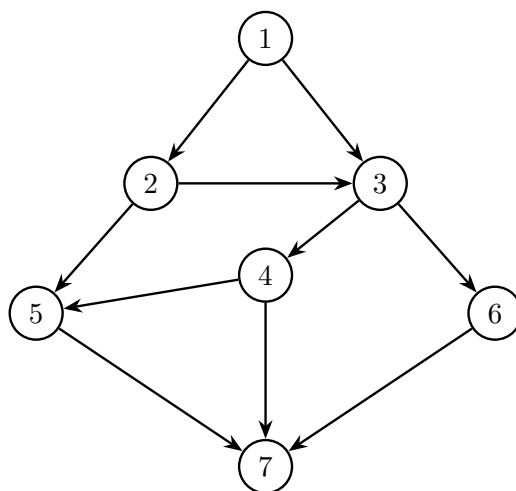
6. Pebble games.

Many two-player strategic games, like the several variants of [Nim](#), can be modeled as follows. There is a directed acyclic graph $G = (V, E)$ presented in “topological” order: the vertices are labeled as $V = \{1, \dots, n\}$, and every edge $(u, v) \in E$ has $u < v$. Moreover, every vertex $i < n$ has at least one outgoing edge.

Two players, called Maize and Blue, take turns in the following game on the graph. They start with a pebble at vertex 1, and Maize plays first. On each turn, the active player (the player whose turn it is) chooses an edge $(u, v) \in E$ and moves the pebble from its current vertex u to a new vertex v . Unless $v = n$, the other player then becomes the active player with the pebble at v . If a player moves the pebble to vertex n (the final one), then that player wins the game.

It turns out that, for any fixed graph G , one player can be *guaranteed* a win by playing “perfectly”—regardless of what their opponent does. In this problem you will develop a dynamic-programming algorithm that, given the graph G as input, determines *which* player can be guaranteed a win, along with *how* that player can play perfectly.

- (2 pts) (a) In the following graph, there are 7 nodes numbered in topological order. Suppose the pebble is at vertex 1. Determine whether or not the active player (the player who is about to move the pebble) can guarantee a win regardless of what the other player does in future turns. Justify your answer.



Solution: Yes, the active player at vertex 1 can guarantee a win. Work backward from vertex 7. At vertices 4, 5, and 6, the active player can move directly to 7 and win, so those vertices are winning. At vertex 3, both possible moves go to winning vertices (4 or 6), so the active player at 3 loses against perfect play. From vertex 1, the active

player moves to 3. The opponent is then at a losing vertex, so no matter whether that opponent moves to 4 or 6, the original player moves next to 7 and wins.

Maize first chooses the edge $1 \rightarrow 3$. Blue's only choices are $3 \rightarrow 4$ and $3 \rightarrow 6$. If Blue chooses $3 \rightarrow 4$, Maize chooses $4 \rightarrow 7$; if Blue chooses $3 \rightarrow 6$, Maize chooses $6 \rightarrow 7$. In either case Maize is the player who moves the pebble to the terminal vertex and therefore wins. This considers every possible response by Blue.

- (2 pts) (b) List all vertices in the above graph for which, if the pebble is at that vertex, the active player can guarantee a win. You do not need to justify your answer.

Solution: The winning vertices are $\{1,2,4,5,6\}$.

Vertex 7 is losing because the game has already ended when the pebble arrives there. Its predecessors 4,5,6 are winning because they can move to 7. Vertex 3 is losing because both of its out-neighbors are winning. Vertex 2 is winning because it can move to the losing vertex 3, and vertex 1 is winning for the same reason. This also verifies every vertex in the displayed set.

- (8 pts) (c) For a given graph, define $W(i) = \text{true}$ if the active player can be guaranteed a win when the pebble is at vertex i . (Otherwise, $W(i) = \text{false}$.) Derive, with justification, a recurrence relation for $W(i)$. Remember to include appropriate base case(s).

Notation: For representing the Boolean operators $\vee, \wedge$ across many variables, you may use the notation $\bigvee, \bigwedge$. For example $\bigwedge_{i=1}^k x_i = x_1 \wedge x_2 \wedge \cdots \wedge x_k$.

Solution: The base case is

$$W(n) = \text{false},$$

because a player whose turn begins with the pebble already at n has no legal move; the previous player has already won by moving to n . For every $i < n$,

$$W(i) = \bigvee_{(i,j) \in E} \neg W(j).$$

In other words, $W(i)$ is true exactly when i has at least one out-neighbor j with $W(j)$ false.

If some out-neighbor j has $W(j)$ false, the active player moves to j . The opponent is then active at a losing position, so the current player can force a win. Conversely, if every out-neighbor is winning for the next active player, then every legal move hands the opponent a winning position. Hence the current player cannot guarantee a win. Since every edge goes from a smaller to a larger label, these values can be computed in decreasing order of vertex label.

Assume that all values $W(j)$ for $j > i$ correctly classify positions whose next active player can force a win. This induction hypothesis applies to every out-neighbor of i . If at least one such j is losing, the active player chooses it and wins by the induction hypothesis. If none is losing, every legal choice gives the opponent a winning position, so no strategy can guarantee a win from i . The base case $i = n$ and this induction step prove the recurrence for all vertices.

- (5 pts) (d) The algorithm should take as input a directed graph $G = (V, E)$ and determine which of the two players has a guaranteed winning strategy. Design a dynamic programming algorithm that runs in $O(m)$ time, where $m = |E|$. Provide pseudocode for the algorithm and justify its running time.

The input graph $G = (V, E)$ is represented via *adjacency lists*, i.e., for each vertex $i \in V$ there is a list of all its *out-neighbors*: vertices j such that $(i, j) \in E$.

Solution: Because the vertices are already in topological order, compute the table from right to left:

```
WinningPlayer(Adj[1..n]):  
    W[n] = false  
    for u = n - 1 downto 1:  
        W[u] = false  
        for each v in Adj[u]:  
            if W[v] = false:  
                W[u] = true  
                break  
  
    if W[1] = true:  
        return "Maize"  
    else:  
        return "Blue"
```

When processing u , every out-neighbor v satisfies $v > u$, so $W[v]$ has already been computed. The inner loops inspect each adjacency-list entry at most once, for a total of m inspections. Initialization and the outer loop take $O(n)$ time. Since every vertex $1, \dots, n - 1$ has an outgoing edge, $m \geq n - 1$, and therefore $O(n + m) = O(m)$.

The final test is made at vertex 1 because Maize is the initially active player. If $W(1)$ is true, the active player at 1, namely Maize, has the guaranteed strategy. If $W(1)$ is false, Maize cannot force a win; since this finite perfect-information game has no draw, Blue must have a guaranteed winning strategy.

- (5 pts) (e) Describe, in clear and precise English, how to extend your algorithm from the previous part so that it also outputs a “winning strategy” for whichever player has one. This should be represented as an array specifying, for each vertex u , where the player should move the pebble if it is at vertex u on that player’s turn.

Solution: While computing W , also maintain an array **Move**. Whenever $W(u)$ is true, select any out-neighbor v with $W(v)$ false and store $\text{Move}[u] = v$. Such a neighbor exists by the recurrence. For a vertex with $W(u)$ false, leave $\text{Move}[u]$ undefined, since the active player has no move that guarantees a win there.

The player who wins from vertex 1 follows **Move** whenever it is that player’s turn. Each prescribed move sends the opponent from a winning vertex to a losing vertex. Any move from a losing vertex sends the other player to a winning vertex, after which the stored move again applies. Eventually a stored move reaches vertex n , so this array is a winning strategy. Storing a neighbor when the first false entry is found adds only

$O(1)$ work per vertex and does not change the $O(m)$ running time.

Under the stored rule, whenever the eventual winner sees a vertex with $W(u) = \text{true}$, it moves to a vertex with value false. The opponent cannot return a false value on its own turn: by the recurrence, every move from a false vertex goes to a true vertex. Thus the winner is always again at a true vertex when its turn returns. Since labels strictly increase, the process terminates, and the winner eventually makes the terminal move.

7. **Spell Checker** Imagine that you are building a spellchecker for a word processor. When the spellchecker encounters an unknown word, you want it to suggest a word in its dictionary that the user might have meant (perhaps they made a typo). One way to generate this suggestion is to measure how “close” the typed word A is to a particular word B from the dictionary, and suggest the closest of all dictionary words. There are many ways to measure closeness; in this problem we will consider a measure known as the *edit distance*, denoted $\text{EDIT-DIST}(A, B)$.

In more detail, given strings A and B , consider transforming A into B , via character *insertions* (i), *deletions* (d), and *substitutions* (s). For example, if $A = \text{ALGORITHM}$ and $B = \text{ALTRUISTIC}$, then one way of transforming A into B is via the following operations:

A	L	G	O	R		I		T	H	M
A	L	T		R	U	I	S	T	I	C
		s	d		i		i		s	s

$\text{EDIT-DIST}(A, B)$ is the minimum cost of transforming string A to string B , given the following three positive numbers:

- c_i , the cost to insert a character;
- c_d , the cost to delete a character;
- c_s , the cost to substitute a character.

In this problem you will give and analyze an efficient dynamic programming algorithm that, on input of two strings A, B , computes $\text{EDIT-DIST}(A, B)$ with the costs for insertion, deletion and substitution as c_i, c_d and c_s respectively.

- (8 pts) (a) Give a recurrence relation for edit distance $\text{EDIT-DIST}(A, B)$ and justify its correctness. Remember to include base case(s).

Hint: the recurrence relation for LCS is a good place to look for inspiration.

Solution: Let $a = |A|$ and $b = |B|$. For $0 \leq i \leq a$ and $0 \leq j \leq b$, let $D(i, j)$ be the minimum cost of transforming the prefix $A[1..i]$ into the prefix $B[1..j]$. To turn a nonempty prefix into the empty string, every character must be deleted, while turning the empty string into a prefix requires one insertion per character. Therefore The base cases are

$$D(0, 0) = 0, \quad D(i, 0) = ic_d \quad (1 \leq i \leq a), \quad D(0, j) = jc_i \quad (1 \leq j \leq b).$$

For $i, j \geq 1$,

$$D(i, j) = \begin{cases} D(i-1, j-1), & A[i] = B[j], \\ \min \begin{cases} D(i-1, j) + c_d, \\ D(i, j-1) + c_i, \\ D(i-1, j-1) + c_s \end{cases}, & A[i] \neq B[j]. \end{cases}$$

For the justification, consider the final operation in an optimal transformation of $A[1..i]$ to $B[1..j]$. If the final characters agree, they need not be changed, and deleting both equal final characters reduces the problem to $(i-1, j-1)$. If they differ, the last operation is exactly one of: delete $A[i]$, insert $B[j]$, or substitute $A[i]$ by $B[j]$. These operations leave the three prefix pairs shown in the recurrence and add their respective costs. The cases are exhaustive, and each candidate can be extended by its stated final operation, so taking the minimum is correct.

In the deletion option, first transform $A[1..i-1]$ into $B[1..j]$, then delete the remaining final character $A[i]$, costing c_d . In the insertion option, first transform $A[1..i]$ into $B[1..j-1]$, then insert $B[j]$, costing c_i . In the substitution option, first transform the shorter prefixes and then replace $A[i]$ by $B[j]$, costing c_s . The boundary values are also forced: turning a nonempty prefix into the empty string requires exactly one deletion per character, and conversely requires one insertion per character.

- (4 pts) (b) Consider the translation of your recurrence into a dynamic programming algorithm. Analyze the running time. While justifying your answer, be sure to include a description of the order in which the DP table should be filled, and where the final answer is located. You are not required to give explicit pseudocode (though you may if you wish).

Solution: Use a table with rows $i = 0, 1, \dots, |A|$ and columns $j = 0, 1, \dots, |B|$. First fill row 0 using $D(0, j) = jc_i$ and column 0 using $D(i, 0) = ic_d$. Then fill the remaining cells in increasing row order, and within each row in increasing column order. When cell (i, j) is filled, the only entries used by its recurrence— $(i-1, j)$, $(i, j-1)$, and $(i-1, j-1)$ —have already been filled. The final answer is

$$D(|A|, |B|).$$

There are $(|A| + 1)(|B| + 1)$ cells and each takes $O(1)$ time to fill, so the running time is $O(|A| |B|)$. The full table uses $O(|A| |B|)$ space.

Each interior cell depends only on its northern, western, and northwestern neighbors. Filling rows from top to bottom and columns from left to right therefore never reads an unfilled entry. If only the numerical distance is needed, two rows suffice and reduce space to $O(|B|)$; the stated full-table bound is appropriate when one also wants to reconstruct an optimal sequence of edit operations.

- (3 EC pts) 8. **Optional extra credit: Wheel deliver.** Jett and Anthony have started a new unicycle-based meal delivery business called “Wheel Deliver” that works as follows. Residents of Ann Arbor submit their requests for a meal each day before noon using the Wheel Deliver App. In the

afternoon, Jett and Anthony cook up a delicious meal in their kitchen and deliver the food on their unicycles.

Let n be the number of customers on a given day and $P = (p_1, \dots, p_n)$ be the list of locations of these customers as points in the 2D plane, sorted in the order in which the customers submitted their requests: p_1 is the location of the first person to place an order, and p_n is the location of the last person to place an order. Let p_0 denote the location of the kitchen from which Jett and Anthony depart. Let $d(p_i, p_j)$ denote the distance between p_i and p_j .

Jett and Anthony wish to split up the list of locations P , not necessarily evenly, so that each delivery is made by exactly one of them. In addition, “Wheel Deliver” has a fairness rule that goes like this: If customers at p_i and p_j for $i < j$ are assigned to the same unicycle rider, then the delivery to the customer at p_i must occur before the delivery to the customer at p_j —even if that makes the trip longer. (If p_i and p_j are assigned to different riders, then we don’t care which delivery occurs first.) Subject to these constraints, the objective is to minimize the total distance travelled by the two unicycle riders.

The referenced figure file is not present in this project; the recurrence below does not depend on the illustration.

Figure 1: Two examples of routes for the two riders. The origin denotes the starting point p_0 . One rider’s route is shown with solid lines and the other with dashed lines. The route on the left is fair because points are visited by increasing indices. The route on the right is not fair because, for example, point p_4 is visited before point p_1 .

Jett and Anthony have hired you to design an efficient algorithm that takes as input an array of points $P = (p_1, p_2, \dots, p_n)$, sorted by the times in which their orders arrived, and outputs the sequences of delivery points for the two riders.

Derive a recurrence relation, including base case(s), that is suitable for an efficient dynamic-programming solution for this problem. Briefly justify its correctness. You do not need to include pseudocode.

Warning: The most natural-seeming recurrences may have subtle errors!

Solution: Let $F(i, j)$ be the minimum total distance after exactly the customers $p_1, \dots, p_j$ have been served, when the two riders are currently at p_i and p_j , where $0 \leq i < j$. Here p_0 is the kitchen, so endpoint p_0 means that this rider has not served a customer yet. We do not need to distinguish the names of the two riders; exchanging their names changes nothing.

For one customer, one rider goes from the kitchen to p_1 and the other stays at the kitchen:

$$F(0, 1) = d(p_0, p_1).$$

Now consider a state with $j \geq 2$.

First suppose $i < j - 1$. Before p_j was served, the rider who is now at p_j had to be at p_{j-1} . The other rider cannot have served p_{j-1} and then ended at p_i , because $i < j - 1$ and each

rider must visit customers in increasing index order. Therefore the last step is forced:

$$p_{j-1} \longrightarrow p_j.$$

Removing this last step leaves the state $F(i, j-1)$, so

$$F(i, j) = F(i, j-1) + d(p_{j-1}, p_j), \quad 0 \leq i < j-1.$$

The remaining case has endpoints p_{j-1} and p_j . Immediately before p_j was served, the other rider was at some p_k with $0 \leq k < j-1$, while one rider was already at p_{j-1} . If the rider at p_k serves p_j , the extra distance is $d(p_k, p_j)$. We try every possible k :

$$F(j-1, j) = \min_{0 \leq k < j-1} \{F(k, j-1) + d(p_k, p_j)\}.$$

These two cases cover every possible last delivery: either the rider at p_{j-1} serves p_j , or the other rider does.

After filling the table, the two riders may end at any p_i and p_n , so the minimum total distance is

$$\min_{0 \leq i < n} F(i, n).$$

To recover the actual routes, store the value of k chosen in each minimum and follow these choices backward from a minimum final state. The first recurrence fills every state with $i < j-1$ in constant time. For the second recurrence, there are $O(n)$ values of j , and for each j we check at most $j-1$ choices of k . The total work is therefore bounded by $2 + 3 + \dots + n = O(n^2)$. The table and parent choices use $O(n^2)$ space.